Expert Systems with Applications 42 (2015) 2371–2381

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

Pruning strategies for mining high utility itemsets

Srikumar Krishnamoorthy *

Indian Institute of Management, Ahmedabad, India

ARTICLE INFO

Article history:
Available online 8 November 2014

Keywords:
High utility itemsets
Frequent itemsets
Data mining

ABSTRACT

High utility itemset mining problem involves the use of internal and external utilities of items (such as profits, margins) to discover interesting patterns from a given transactional database. It is an extension of the basic frequent itemset mining problem and is proven to be considerably hard and intractable. This is due to the lack of inherent structural properties of high utility itemsets that can be exploited. Several heuristic methods have been suggested in the literature to limit the large search space. This paper aims to improve the state-of-the-art and proposes a high utility mining method that employs novel pruning strategies. The utility of the proposed method is demonstrated through rigorous experimentation on several real and synthetic benchmark sparse and dense datasets. A comparative evaluation of the method against a state-of-the-art method is also presented. Our experimental results reveal that the proposed method is very effective in pruning unpromising candidates, especially for sparse transactional databases.
© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

High utility itemset mining is one of the important problems that has received significant attention in the last several years. This is largely due to its potential applicability in several business and scientific applications. A high utility mining system provides flexibility to a decision maker to incorporate her/his notion of item utilities (profit, margin and so on) into the itemset mining process. As a result, the discovered patterns are highly likely to be of interest to the decision maker. On the other hand, traditional frequent set mining (Agrawal et al., 1994) methods largely rely on item frequencies. The high utility mining, therefore, can be considered as an extension or generalized version of frequent set mining. While the former method uses a generalized utility function, the latter method uses item support or frequencies as the utility function during the mining process.

Frequent set mining methods leverage the anti-monotonic property of support for efficient mining. However, a high utility itemset do not satisfy anti-monotone property (Liu, Liao, & Choudhary, 2005; Yao & Hamilton, 2006). This makes the high utility mining problem considerably hard and intractable. Several algorithms for high utility mining have been proposed in the literature. These algorithms can be broadly classified as level-wise candidate generation and test approach (Liu et al., 2005; Li, Yeh, & Chang, 2008; Yao & Hamilton, 2006), tree-based approach (Ahmed, Tanbeer, Jeong, & Lee, 2009; Ahmed, Tanbeer, Jeong, & Lee, 2011; Tseng, Shie, Wu, & Yu, 2012) and depth-first approach (Liu & Qu, 2012). Most of these algorithms use the concept of *transaction weighted utility* (Liu et al., 2005) to minimize the number of utility computations made during the mining process. However, the *transaction weighted utility* is known to overestimate the true utility of an itemset. This leads to a lot of wasted utility computations for itemsets that eventually do not satisfy the minimum utility threshold. This paper aims to address the foregoing limitations with the help of a new high utility mining system that employs several pruning strategies.

The key contributions of this paper are as follows: We present a method for efficiently discovering high utility itemsets. The proposed method employs two novel pruning strategies, namely partitioned utility pruning and lookahead utility pruning. We demonstrate the usefulness of the proposed method through rigorous experimental evaluation on several real and synthetic benchmark sparse and dense datasets. Furthermore, we present comparative evaluation of the method against a state-of-the-art utility mining system and report our findings.

The rest of the paper is organized as follows. Section 2 describes the related work on high utility mining. Section 3 defines the problem and describes the key definitions and notations used in this paper. Section 4 presents the proposed data structure and pruning strategies. Section 5 outlines our high utility mining method. Subsequently, Section 6 provides detailed experimental analysis and results. Finally, Section 7 gives concluding remarks.

* Tel.: +91 79 6632 4834.
E-mail address: srikumark@iimahd.ernet.in

http://dx.doi.org/10.1016/j.eswa.2014.11.001
0957-4174/© 2014 Elsevier Ltd. All rights reserved.

2. Related literature

Liu et al. (2005) use the concept of *transaction weighted utility* (TWU) for mining high utility itemsets. Their two-phase algorithm mines high utility itemsets in a two step process. In the first step, the algorithm exploits the anti-monotonic property of TWU of itemsets to mine all high TWU itemsets. Then, in the second step, actual utilities of itemsets are computed and low utility itemsets are discarded. The algorithm suffers from scalability issues due to its iterative level-wise candidate generation and test methodology.

UMining and UMining_H algorithms proposed by Yao and Hamilton (2006) utilizes two pruning strategies, namely utility upper bound and support upper bound. While the former strategy is guaranteed to generate all itemsets, the latter strategy is heuristically oriented and may erroneously prune genuine itemsets. The pruning properties exploited by the algorithm require a level-wise mining. Therefore, it may not be easily translatable to more efficient depth oriented utility mining approaches. It also suffers from scalability issues due to its level-wise candidate generation, prune and test methodology. FUM and DCG+ (Li et al., 2008) are level-wise utility mining algorithms that use an isolated items discarding strategy (IIDS) to limit the number of candidates generated and tested.

GPA algorithm (Lan, Hong, & Tseng, 2012) presents an extension to the basic concepts of Two-phase algorithm (Liu et al., 2005). The algorithm follows a level wise mining approach and iteratively generates a tighter utility upper bound. It removes unwanted items iteratively before performing utility computation to determine a tighter upper bound. In addition, the algorithm employs a transaction size reduction strategy to combine duplicate transactions generated during the mining process. The authors demonstrate that GPA algorithm performs much better than a Two-phase algorithm both on pruning and execution efficiency.

PB algorithm (Lan, Hong, & Tseng, 2014) uses special indexing structures and projection based methods to efficiently mine high utility itemsets. The authors demonstrate that PB algorithm is much more computationally efficient compared to Two-phase and CTU-PRO (Erwin, Gopalan, & Achuthan, 2007).

Tree based algorithms that mine high utility itemsets without expensive candidate generation and test methodology include IHUP (Ahmed et al., 2009), HUC-Prune (Ahmed et al., 2011), and UP-Growth+ (Tseng et al., 2012). IHUP (Ahmed et al., 2009) is experimentally proven to be better than Two-phase (Liu et al., 2005), FUM (Li et al., 2008), and DCG+ (Li et al., 2008).

HUI-Miner is one of the recent and most efficient depth-first algorithms proposed by Liu and Qu (2012). The authors introduced a new data structure called utility lists which is similar to the tid lists used in Eclat algorithm for mining frequent itemsets (Zaki, 2000). The information captured in utility lists during the mining process is exploited to limit the overall search space. The authors demonstrate that their method is superior to IHUP (Ahmed et al., 2009; Tseng, Wu, Shie, & Yu, 2010) and UP-Growth+ (Tseng et al., 2012). The authors show almost two orders of magnitude improvement over other methods in the literature.

Liu, Wang, and Fung (2012) proposed d^2HUP algorithm for efficiently mining high utility itemsets. The authors used the concepts of irrelevant item filtering and lookahead pruning for efficient mining. The concept of irrelevant item filtering involves iteratively eliminating the irrelevant items from the utility computation process. d^2HUP (Liu et al., 2012) and HUI-Miner (Liu & Qu, 2012) algorithms were introduced by different sets of authors at almost around the same time. However, one can observe that the key pruning strategies adopted by both of these algorithms are conceptually equivalent though the underlying data structures used are quite different. d^2HUP also uses the concept of lookahead strategy for efficient utility mining in the case of dense datasets. It is to be noted that the final high utility itemsets generated (with lookahead pruning) are not complete. An additional iteration of the generated itemsets will be required to enumerate the actual high utility itemsets and their utility values. The authors demonstrate that their algorithm is up to an order of magnitude faster than UP-Growth (Tseng et al., 2010).

An interesting extension to the basic utility mining problem is the high on-shelf utility mining problem. This problem was introduced by Lan, Hong, and Tseng (2011). The authors design a periodic total transaction utility table and a new pruning strategy based on on-shelf utility measure. The proposed method is shown to be better than a traditional high utility itemset mining on synthetic benchmark datasets.

This research work builds on the extant literature and presents new pruning strategies for efficiently mining high utility itemsets.

3. Definition and notation

We formally define the key terms in utility mining using the standard conventions followed in the literature (Ahmed et al., 2009; Liu & Qu, 2012; Liu et al., 2005; Yao & Hamilton, 2006).

Let $I = \{i_1, i_2 \ldots i_m\}$ be a set of distinct items. A set $X \subseteq I$ is called an itemset. A transaction $T_j = \{x_l | l = 1, 2 \ldots N_j, x_l \in I\}$, where N_j is the number of items in transaction T_j. A transaction database D has set of transactions, $D = \{T_1, T_2 \ldots T_n\}$, where n is the total number of transactions in the database. A sample transaction database D is given in Table 1.

Definition 1. Each item $x_i \in I$ is assigned an external utility value (e.g. profit), referred as $EU(x_i)$. For example, in Table 1, $EU(B) = 2$.

Definition 2. Each item $x_i \in T_j$ is assigned an internal utility value, referred as $IU(x_i, T_j)$. For example, in Table 1, $IU(B, T_1) = 1$.

Definition 3. The utility of an item $x_i \in T_j$, denoted as $U(x_i, T_j)$ is computed as the product of external and internal utilities of item in the transaction, T_j. That is,

$$U(x_i, T_j) = EU(x_i) * IU(x_i, T_j) \quad (1)$$

For example, in Table 1, $U(B, T_1) = EU(B) * IU(B, T_1) = 2 * 1 = 2$.

Definition 4. The utility of an itemset X in transaction T_j ($X \subseteq T_j$) is denoted as $U(X, T_j)$.

$$U(X, T_j) = \sum_{x_i \in X} U(x_i, T_j) \quad (2)$$

For example, in Table 1, $U(ABD, T_1) = 13$.

Table 1
Purchase history.

TID	Transaction	Purchase qty (IU)	Utility (U)	Transaction utility (TU)
T_1	A, B, D, F, G	1, 1, 2, 1, 2	1, 2, 10, 3, 2	18
T_2	D, H	1, 1	5, 2	7
T_3	A, B, F, G	2, 2, 2, 1	2, 4, 6, 1	13
T_4	B, C, D, E, F	1, 1, 1, 2, 2	2, 1, 5, 8, 6	22
T_5	D, F	2, 1	10, 3	13
T_6	B, C, E, F	1, 1, 2, 1	2, 1, 8, 3	14
T_7	A, G	3, 2	3, 2	5
T_8	C, E, G	2, 1, 3	2, 4, 3	9
T_9	A, B, C, D, E, G	2, 1, 2, 1, 1, 3	2, 2, 2, 5, 4, 3	18
T_{10}	B, D, E, G	2, 1, 2, 2	4, 5, 8, 2	19
T_{11}	F, G, I	1, 1, 1	3, 1, 1	5
T_{12}	B, C	1, 5	2, 5	7

Definition 5. The support of an itemset X in database D is denoted as $Sup(X)$. It is the ratio of the frequency of occurrence of the itemset X in D divided by the total number of transactions (n). For example, $Sup(AB) = 3/12 = 25\%$.

Definition 6. The utility of an itemset X in database D is denoted as $U(X)$.

$$U(X) = \sum_{X \subseteq T_j \in D} U(X, T_j) \tag{3}$$

For example, $U(ABD) = U(ABD, T_1) + U(ABD, T_9) = 22$.

Definition 7. The transaction utility, $TU(T_j)$ for transaction T_j is computed as

$$TU(T_j) = \sum_{X \subseteq T_j \text{ and } x_i \in X} U(x_i, T_j) \tag{4}$$

For example, $TU(T_6) = U(B, T_6) + U(C, T_6) + U(E, T_6) + U(F, T_6) = 2 + 1 + 8 + 3 = 14$.

Definition 8. The user specified minimum utility percentage value is denoted as δ. The absolute minimum utility value, denoted as *minutil* is then computed as

$$minutil = \delta * \sum_{T_j \in D} TU(T_j) \tag{5}$$

For the illustrative example in Table 1, when $\delta = 24\%$, the absolute minimum utility value, $minutil = 0.24 * 150 = 36$.

Definition 9. The transaction-weighted utility of an itemset X, denoted as $TWU(X)$, is defined by

$$TWU(X) = \sum_{X \subseteq T_j \in D} TU(T_j) \tag{6}$$

For the transaction database in Table 1, $TWU(A) = 18 + 13 + 5 + 18 = 54$.

TWU Pruning Property. If $TWU(X) < minutil$, then $\forall X' \supseteq X, TWU(X') \leqslant TWU(X) < minutil$.

As per the apriori property, $Sup(X') \leqslant Sup(X)$. This implies that $TWU(X') \leqslant TWU(X) < minutil$. This property is commonly exploited as a key pruning strategy for mining high utility itemsets in the literature (Ahmed et al., 2009; Liu et al., 2005).

Definition 10. Let T_j/X denote the set of all items after X in T_j. For example, in Table 4, $T_1/AG = DB, T_4/CE = FDB$

Definition 11. The remaining utility of an itemset X in transaction $T_j (X \subseteq T_j)$ is denoted as $RU(X, T_j)$ and is computed as,

$$RU(X, T_j) = \sum_{x_i \in (T_j/X)} U(x_i, T_j), \tag{7}$$

For example, in Table 4, $RU(AG, T_1) = 10 + 2 = 12, RU(CEB, T_6) = 0$.

Definition 12. The remaining utility of an itemset X in database D is denoted as RU(X).

$$RU(X) = \sum_{X \subseteq T_j \in D} RU(X, T_j) \tag{8}$$

For example, in Table 4, $RU(AG) = 12 + 4 + 0 + 7 = 23$.

Definition 13. The database $D = \{T_1, T_2 \ldots T_n\}$ can be considered as K non-overlapping sequential set of partitions, $P = \{P_1, P_2 \ldots P_k\}$. Each partition, $P_k \subseteq D$ has a block of transactions. The size of each block is computed as $\lceil n/K \rceil$.

For example, in Table 4, assuming $K = 4$ with $n = 12$ yields a block size of 3. Therefore, $P = \{P_1, P_2, P_3, P_4\} = \{\{T_1, T_2, T_3\}, \{T_4, T_5, T_6\}, \{T_7, T_8, T_9\}, \{T_{10}, T_{11}, T_{12}\}\}$.

Definition 14. The utility of an itemset in a partition, P_k is denoted as $UP(X, P_k)$.

$$UP(X, P_k) = \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} U(X, T_j) \tag{9}$$

In Table 4, assuming $K = 4, UP(AG, P_1) = 3 + 0 + 3 = 6$, $UP(AG, P_2) = 0, UP(AG, P_3) = 5 + 0 + 5 = 10$ and $UP(AG, P_4) = 0$.

Lemma 1. $U(X) = \sum_{\forall P_k \in P} UP(X, P_k)$.

Proof. $U(X) = \sum_{\forall P_k \in P} UP(X, P_k) = \sum_{\forall P_k \in P} \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} U(X, T_j)$. $= \sum_{X \subseteq T_j \in D} U(X, T_j)$, which is true by definition 6.

Accumulation of $U(X, T_j)$ over all transactions in individual partitions and then over all partitions is equivalent to accumulation of $U(X, T_j)$ over all transactions in the database. Hence the proof.

For example, $U(AG) = UP(AG, P_1) + UP(AG, P_2) + UP(AG, P_3) + UP(AG, P_4) = 6 + 0 + 10 + 0 = 16$. □

Definition 15. The remaining utility of an itemset X in database partition P_k is denoted as $RUP(X, P_k)$.

$$RUP(X, P_k) = \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} RU(X, T_j) \tag{10}$$

In Table 4, assuming $K = 4, RUP(AG, P_1) = 12 + 0 + 4 = 16$, $RUP(AG, P_2) = 0, RUP(AG, P_3) = 0 + 0 + 7$ and $RUP(AG, P_4) = 0$.

Lemma 2. $RU(X) = \sum_{\forall P_k \in P} RUP(X, P_k)$.

Proof. $RU(X) = \sum_{\forall P_k \in P} RUP(X, P_k) = \sum_{\forall P_k \in P} \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} RU(X, T_j)$. $= \sum_{X \subseteq T_j \in D} RU(X, T_j)$, which is true by definition 12.

Accumulation of $RU(X, T_j)$ over all transactions in individual partitions and then over all partitions is equivalent to accumulation of $RU(X, T_j)$ over all transactions in the database. Hence the proof.

For example, $RU(AG) = RUP(AG, P_1) + RUP(AG, P_2) + RUP(AG, P_3) + RUP(AG, P_4) = 16 + 0 + 7 + 0 = 23$. □

Lemma 3. *If* $UP(X, P_k) = 0$, *then for all* $X' \supseteq X, UP(X', P_k) = 0$.

Proof. $UP(X, P_k) = \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} U(X, T_j) = 0$. This implies that none of the transactions T_j in P_k contains X. As $X' \supseteq X$, none of the T_js in P_k contains X' as well. That is, $\sum_{\forall T_j \in P_k \text{and} X' \subseteq T_j} U(X', T_j) = UP(X', P_k) = 0$. Hence, the proof. □

Corollary 1. *If* $UP(X, P_k) = 0$, *then* $RUP(X, P_k) = 0$ *and* $RUP(X', P_k) = 0$ *for all* $X' \supseteq X$.

Proof. $RUP(X, P_k) = \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} RU(X, T_j) = \sum_{\forall T_j \in P_k \text{ and } X \subseteq T_j} U(T_j/X)$. As X is not present in any of the transactions, the remaining utility of items that comes after $X (or X')$ is also zero. Hence, the proof. □

Lemma 4. *Given two itemsets X and Y,* $\sum_{UP(Y,P_k)=0} UP(XY,P_k) + RUP(XY,P_k) = 0$.

Proof. The constraint $UP(Y,P_k)=0$ indicates that itemset Y is not present in any of the transactions in T_j in P_k. As per Lemma 3, any superset of Y (like XY) also cannot be present in any of the transactions T_j in P_k. Therefore, $UP(XY,P_k)=0$ when $UP(Y,P_k)=0$. Similarly, $RUP(XY,P_k)$ can be verified to be zero as per Corollary 1. Hence, the proof. □

Definition 16. The problem of high utility mining involves determining all itemsets in D whose utility values are greater than or equal to the user defined minimum utility value *minutil*. That is,

$$HUI = \{X : U(X) | X \subseteq I, U(X) \geqslant minutil\} \quad (11)$$

For the transaction database in Tables 1 and 4, when $minutil = 36, HUI = \{BDE : 43, BE : 38, DF : 37, D : 40\}$.

Our main objective in this paper is to mine high utility itemsets (*HUI*) from transaction database, D, for a given δ or *minutil*.

4. Data structure and key pruning strategies

We describe our new data structure and key pruning strategies in this section. The partitioned utility list data structure introduced in this paper borrows basic ideas from tid list representation used by Zaki (2000). It is also an extension of utility list data structure proposed by Liu and Qu (2012).

Definition 17. A *PUL* of an itemset X, denoted as $PUL(X)$ is a new data structure that contains a tidlist and a partition list. Each tuple or entry in the tidlist is a triple $<TID, U(X,T_j), RU(X,T_j)>$. Each entry in the partition list is a pair $<P_k, UP(X,P_k)+RUP(X,P_k)>$. For example, the initial 1-itemset *PUL*s for the processed transaction database in Table 4 is given in Fig. 1 (assuming $K=4$).

The proposed method employs four key pruning strategies namely, TWU prune, U-Prune, PU-Prune and LA-Prune for efficiently mining *HUI*s. The TWU prune property commonly used in the literature was discussed earlier (refer to Definition 9).

U-Prune Property If the total sum of the utility and remaining utility of an itemset X is less than *minutil*, then none of the supersets of X is a high utility itemset. That is, if $U(X)+RU(X) < minutil$, then $X' \notin HUI \forall X' \supseteq X$. This utility property was first introduced and effectively used as a pruning strategy for high utility mining in HUI-Miner (Liu & Qu, 2012).

Lemma 5. (PU-Prune) *For any given two itemsets X and Y, if* $\sum_{UP(Y,P_k)>0} UP(X,P_k) + RUP(X,P_k) < minutil$, *then* $\forall Y' \supseteq Y$ *and* $X' \supseteq X, X'Y' \notin HUI$. *In essence, if the conditional sum in the lemma is true, none of the supersets of X that contain Y belong to HUI.*

Proof. $U(X'Y',T_j) = U(XY,T_j) + U(X'Y' - XY,T_j) = U(XY,T_j) + U(X'Y'/XY,T_j) \leqslant U(XY,T_j) + RU(XY,T_j)$, as $XY \subseteq X'Y' \subseteq T_j$

$$\begin{aligned}
U(X'Y') &= \sum_{X'Y' \subseteq T_j \in D} U(X'Y',T_j) = \sum_{\forall P_k \in P} \sum_{\forall T_j \in P_k \text{ and } X'Y' \subseteq T_j} U(X'Y',T_j) \\
&\leqslant \sum_{\forall P_k \in P} \sum_{\forall T_j \in P_k \text{ and } XY \subseteq T_j} U(XY,T_j) + RU(XY,T_j) \\
&\leqslant \sum_{\forall P_k \in P} UP(XY,P_k) + RUP(XY,P_k), \text{ by definitions 14 and 15} \\
&\leqslant \sum_{UP(Y,P_k)=0} [UP(XY,P_k) + RUP(XY,P_k)] + \sum_{UP(Y,P_k)>0} [UP(XY,P_k) \\
&\quad + RUP(XY,P_k)], \text{ by expansion,} \\
&\leqslant \sum_{UP(Y,P_k)>0} [UP(XY,P_k) + RUP(XY,P_k)], \text{ as per Lemma 4} \\
&\leqslant \sum_{UP(Y,P_k)>0} [UP(X,P_k) + RUP(X,P_k)], \text{ as } X \subset XY \subseteq T_j \in P_k \\
&< minutil, \text{ as per original lemma condition}
\end{aligned}$$

Therefore, $U(X'Y') \notin HUI$ and hence the proof. □

The following three inferences can be deduced if the conditional statement in the above lemma is true: (a) $XY \notin HUI$, (b) $XY' \notin HUI$. and (c) $X'Y' \notin HUI$. Inferences (a) and (b) allow pruning XY and all of its descendants. Inference (c) allows pruning different sub-parts

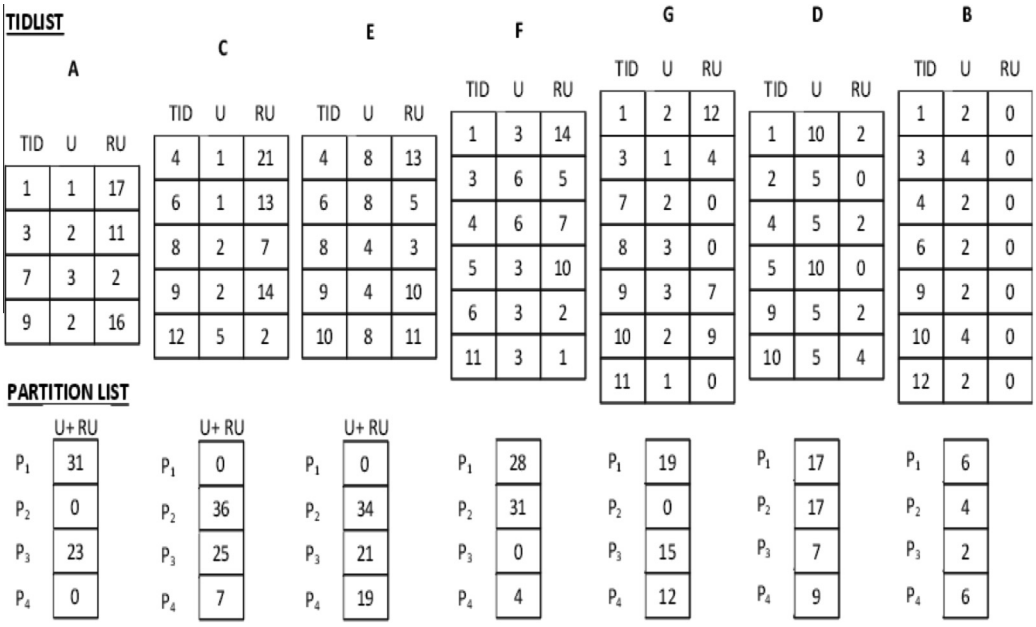


Fig. 1. Partitioned utility lists for 1-itemset.

of the tree without unnecessary candidate evaluations. We exploit these key inferences from the pruning property to limit the search space of utility mining.

An Illustrative Example. For the transaction database in Table 4, a sample exploration path is given in Fig. 2. Let us assume $minutil = 36$. Each node X in the tree is marked with its utility $U(X)$. Let us consider, node or itemset C and its extensions in the search tree. As per Fig. 1, the node Cs, $U(C) = 11$ and $RU(C) = 57, U(C) + RU(C) = 68$. Applying U-Prune property, $U(C) + RU(C) = 68 > minutil$, fails to prune C.

Let us explore the extensions of node C. Consider node CG which can be generated by combining node C and node G. Applying PU-Prune property $\sum_{UP(Y,P_k)>0} UP(X,P_k) + RUP(X,P_k) < minutil$ or $0 + 25 + 7 < 36$ evaluates to true. Therefore, node CG and it's descendants are pruned without actually constructing the node CG's PUL. Also, node $CEG, CEFG, CGD$ are not generated as the node CG is pruned.

Lemma 6. (LA-Prune) *Given two itemsets X and Y, if* $\sum_{\forall T_i \in D} U(X,T_i) + RU(X,T_i) - \sum_{\forall T_j \in D, X \subseteq T_j \text{ and } Y \not\subseteq T_j} U(X,T_j) + RU(X,T_j) < minutil$, *then* $\forall Y' \supseteq Y$ *and* $X' \supseteq X, X'Y' \notin HUI$,.

Proof. There are two parts in the left hand side of the above lemma condition. The first part sums up to $U(X) + RU(X)$. It computes the total utility value of X from transactions where X is present (irrespective of whether Y is present or not). The second part subtracts the itemset X's utility values of transactions where the itemset X is present but not Y. The cumulative sum of the two parts gives a tighter utility upper bound for transactions where XY is likely to be present. If the resulting utility upper bound is less than $minutil$, then neither the itemset XY (nor any of its descendants) can be part of HUI. Furthermore, the utility value computations are made with reference to X and hence none of the descendants of X that contain Y can be part of HUI.

Mathematically, $\sum_{\forall T_i \in D} U(X,T_i) + RU(X,T_i) - \sum_{\forall T_j in D, X \subseteq T_j \text{ and } Y \not\subseteq T_j} U(X,T_j) + RU(X,T_j) < minutil$, expanding the first term in the left side of expression by rewriting the condition (without loss of generality) yields:

$$\sum_{\forall T_i \in D, X \subseteq T_i \text{ and } Y \subseteq T_i} U(X,T_i) + RU(X,T_i) + \sum_{\forall T_i \in D, X \subseteq T_i \text{ and } Y \not\subseteq T_i} U(X,T_i) + RU(X,T_i) - \sum_{\forall T_j, X \subseteq T_j \text{ and } Y \not\subseteq T_j} U(X,T_j) + RU(X,T_j) < minutil$$

Cancelling out the 2nd and 3rd terms (which are equal) in the above equation gives:

$$\sum_{\forall T_i \in D, X \subseteq T_i and Y \subseteq T_i} U(X,T_i) + RU(X,T_i) < minutil \quad (12)$$

$U(X'Y') = \sum_{X'Y' \subseteq T_i \in D} U(X'Y', T_i) \leqslant \sum_{XY \subseteq T_i \in D} U(XY,T_i) + RU(XY,T_i)$, using the inference from the proof of Lemma 5 $\leqslant \sum_{\forall T_i \in D, X \subseteq T_i \text{ and } Y \subseteq T_i} U(XY,T_i) + RU(XY,T_i)$, rewriting the conditions without loss of generality $\leqslant \sum_{\forall T_i \in D, X \subseteq T_i \text{ and } Y \subseteq T_i} U(X,T_i) + RU(X,T_i)$, as $X \subset XY \subseteq T_i < minutil$, as per Eq. 12.

Therefore, $U(X'Y') \notin HUI$ and hence the proof. □

LA-Prune property provides a tighter utility upper bound for any itemset X that contains Y. It is applied while constructing PUL for XY from individual PULs of X and Y.

An Illustrative Example. Let us consider node EGD which is generated by combining nodes $X = EG$ and $Y = ED$. It can be easily verified that $U(EG) + RU(EG) = 24 + 16 = 40$. Neither U-Prune nor PU-Prune property is satisfied in this case. Therefore, PUL for node EGD is constructed. During the PUL construction process, the algorithm evaluates if LA-Prune property is satisfied.

The itemset EG first appears in transaction id 8 (refer to Fig. 1), whereas ED first appears in transaction id 9. Hence, the utility values for transaction 8 can be subtracted from the overall sum $U(EG) + RU(EG)$. As, $U(EG, T_8) + RU(EG, T_8) = 7 + 0 = 7$, the value of $U(EG) + RU(EG) - U(EG, T_8) - RU(EG, T_8)$ gives $40 - 7 = 33 < minutil$. The PUL construction process for node EGD can be terminated without traversing remaining transactions in D as the LA-Prune property is satisfied.

The subtree node $EGDB$ is not generated since EGD and EGB were pruned. It is to be noted that the node $EGDB$ will not be generated even if the node EGB alone is pruned.

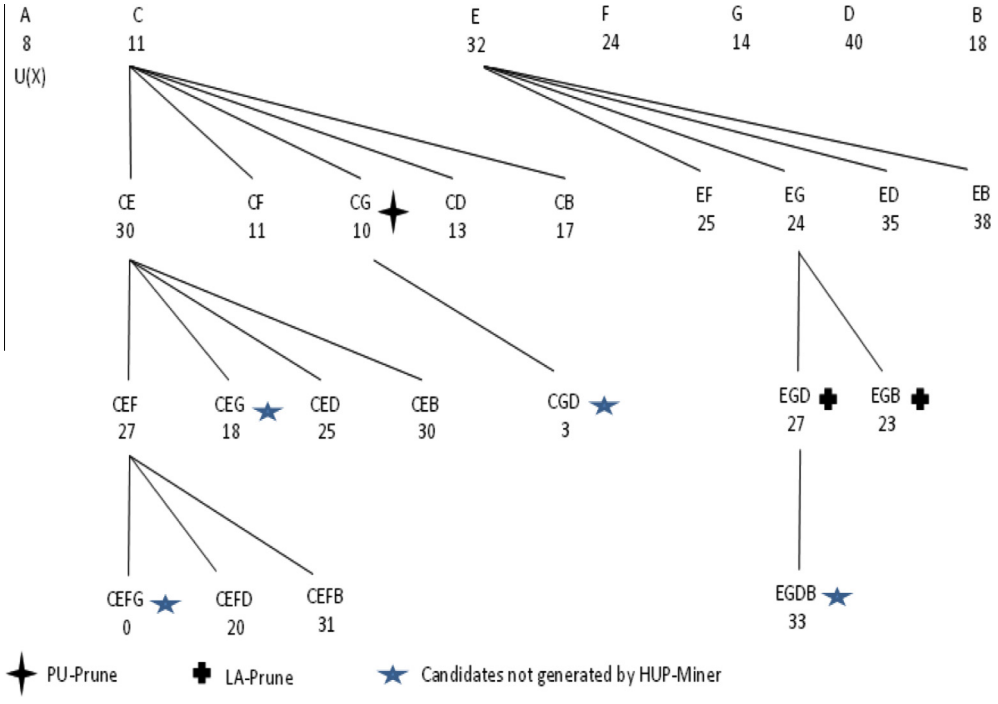


Fig. 2. A sample tree with illustration for different pruning strategies.

Algorithm 1. HUP-Miner: Main

Input: D, the transaction database, *minutil*, the minimum utility threshold, K, user defined number of partitions for utility lists
Output: *HUI*, the set of all high utility itemsets
1. Scan D and Compute *TWU* for all 1-itemsets
2. **for** each $T_j \in D$ **do**
3. Sort x_i's in T_j in *TWU* ascending order
4. $TU_j \leftarrow 0$ //*Recompute TU*
5. **for** each item $x_i \in T_j$ **do**
6. **if** $TWU(x_i) \geqslant minutil$ **then** // *TWU Pruning*
7. $TU_j = TU_j + U(x_i, T_j)$
8. **end if**
9. **end for**
10. Iteratively construct 1-itemset *PULs* with K partitions
11. **end for**
12. HUI = Explore-Search-Tree({},*PULs*, *minutil*) //**Refer Algo 1B**

5. Our proposed method

The proposed method–hereinafter referred as HUP-Miner–uses a new partitioned utility list data structure for efficiently mining *HUIs*. Some of the key steps of our algorithm are described next. The complete pseudo-code for HUP-Miner is given in HUP-Miner (Algorithm 1), Explore-Search-Tree (Algorithm 1B) and Construct-PUL (Algorithm 1C). HUP-Miner utilizes four key pruning strategies described earlier for efficient mining of *HUIs*. *TWU* pruning property is utilized during the initial mining process. The other three pruning properties (U-Prune, PU-Prune, and LA-Prune) are exploited during the recursive search space exploration.

Algorithm 1B. HUP-Miner: Explore-Search-Tree

Input: R, the *PUL* of itemset R, *PULs*, the set of *PULs* of all R's 1-extensions, *minutil*, the minimum utility threshold
Output: all *HUIs* with prefix R
1. **for** each partitioned utility list X in *PULs* **do** // *Explore Search Tree*
2. **if** $U(X) \geqslant minutil$ **then** $HUI = \{HUI \cup X\}$
3. **if** $U(X) + RU(X) \geqslant minutil$ **then** // *U-Prune*
4. $exPULs \leftarrow \{\}$
5. **for** each partitioned utility list Y after X in *PULs* **do**
6. **if** PU-Prune(X, Y) **then continue** to next Y // *PU-Prune*
7. $PUL(XY)$ = ConstructPUL(R, X, Y) // **Refer Algorithm 1C**
8. **if** $PUL(XY) \neq$ NULL **then**
$exPULs$ = $\{exPULs \cup PUL(XY)\}$
end if
9. **end for**
10. Explore-Search-Tree($X, exPULs, minutil$)
11. **end if**
12. **end for**

5.1. TWU computation & partitioned utility list generation

HUP-Miner makes an initial database scan to calculate the *TWU* of 1-itemsets (step 1 of Algorithm 1). The algorithm then iterates through individual transactions in D to sort items, recompute TU_j by applying *TWU* Pruning property and construct *PULs* for 1-itemsets (steps 2 to 11 of Algorithm 1).

For example, let us assume $minutil = 36$ and $K = 4$ for the sample data given in Tables 1 and 2. Applying the above HUP-Miner steps gives the results as shown in Tables 3 and 4. The initial *PULs* created for the 1-itemsets for the running example is shown in Fig. 1.

Subsequently, the algorithm makes use of the generated 1-itemset *PULs* to explore the search space for mining all the *HUIs* from D.

Algorithm 1C. HUP-Miner: ConstructPUL (*PUL*)

Input: *PUL* of itemset R, Rx and Ry
Output: *PUL* of itemset Rxy
1. PUL(Rxy) = NULL
2. **set** TOTAL_UTILITY(Rx) = U(Rx)+RU(Rx)
3. **for** each element Ex $\in$ Rx **do**
4. **if** $\exists$ Ey $\in$ Ry and Ex.TID==Ey.TID **then**
5. **if** R is not empty **then**
6. **find** E in Rx such that E.TID==Ex.TID
7. Exy=<Ex.TID,U(Rx,Ex.TID)+U(Ry,Ey.TID)-U(R,E.TID), RU(Ry,Ey.TID)>
8. **else**
9. Exy=<Ex.TID,U(Rx,Ex.TID)+U(Ry,Ey.TID),RU (Ry,Ey.TID)>
10. **end if**
11. PUL(Rxy) = {PUL(Rxy) $\cup$ Exy} // *Insert or update tidlist*
12. Create or update partition list of PUL(Rxy)
13. **else** // *Recompute TOTAL_UTILITY and apply LA-Prune*
14. TOTAL_UTILITY(Rx)=TOTAL_UTILITY(Rx)-U(Rx,Ex.TID)-RU(Rx,Ex.TID)
15. **if** TOTAL_UTILITY(Rx) < *minutil* **then return** NULL
16. **end if**
17. **end for**
18. **return** PUL(Rxy)

5.2. Tree exploration process

HUP-Miner explores the search tree recursively by using the partitioned utility lists. For a database with m items, an exhaustive search has to explore 2^m itemsets which is excessively time consuming. Therefore, the proposed method exploits three key pruning strategies during tree exploration to limit the search space. More specifically, the pruning strategies used during the recursive mining process include U-Prune (Liu & Qu, 2012), PU-Prune and LA-Prune. The key steps of the exploration process are described in Algorithm 1B.

In each pass of the algorithm, an itemset X (and its complete subtree) is pruned if it satisfies the U-Prune property (step 3 of Algorithm 1B). An unpruned itemset X is combined with each of it's child nodes (Y) in the search path. The algorithm then evaluates PU-Prune property for itemset XY (step 6 of Algorithm 1B). If PU-Prune property is not satisfied, *PUL* for the itemset XY is constructed as described in Algorithm 1C. A successfully constructed *PUL* is accumulated (step 8 of Algorithm 1B) for

Table 2
Item profits.

Item	A	B	C	D	E	F	G	H	I
Profit $ per unit (EU)	1	2	1	5	4	3	1	2	1

Table 3
Transaction weighted utility.

Item	A	C	E	F	G	D	B
TWU	54	70	82	84	86	95	111

Table 4
Transaction with items ordered by TWU.

TID	Transaction	Utility (U)	TU	PID
T_1	A, F, G, D, B	1, 3, 2, 10, 2	18	P_1
T_2	D	5	5	P_1
T_3	A, F, G, B	2, 6, 1, 4	13	P_1
T_4	C, E, F, D, B	1, 8, 6, 5, 2	22	P_2
T_5	F, D	3, 10	13	P_2
T_6	C, E, F, B	1, 8, 3, 2	14	P_2
T_7	A, G	3, 2	5	P_3
T_8	C, E, G	2, 4, 3	9	P_3
T_9	A, C, E, G, D, B	2, 2, 4, 3, 5, 2	18	P_3
T_{10}	E, G, D, B	8, 2, 5, 4	19	P_4
T_{11}	F, G	3, 1	4	P_4
T_{12}	C, B	5, 2	7	P_4

subsequent tree exploration (step 10 of Algorithm 1B). The above process (steps 5–9 of Algorithm 1B) is repeated for all *Y* nodes that are children of *X*.

The foregoing steps are repeated in a recursive manner (step 10 of Algorithm 1B) until all of the *HUI*s are mined. During the recursive mining process, all itemsets *X* that satisfy *minutil* constraint are stored in *HUI* (step 2 of Algorithm 1B).

5.3. Partitioned utility list (PUL) construction process

The partitioned utility list for itemset *Rxy* is computed by performing tid list intersection of itemsets *Rx* and *Ry*. This process is quite similar to HUI-Miner (Liu & Qu, 2012). The key distinctive feature of the proposed *PUL* construction process is the use of LA-Prune property to prune unpromising candidates (steps 2,11–12,14–15 of Algorithm 1C). In addition, HUP-Miner creates a partition list for each itemset during the *PUL* construction process. An illustrative example of actual partition list information maintained for an itemset can be referred in Fig. 1.

LA-Prune property provides a tighter utility upper bound for any itemset *X* that contains *Y*. It is applied while constructing *PUL*(*Rxy*) from the individual *PUL*s of *Rx* and *Ry*. The pruning property is evaluated to check if a transaction T_i that contains *Rx* but not *Ry*. If this check is found to be true (steps 13–16 of Algorithm 1C), the algorithm subtracts the utility values of the relevant transaction $(U(Rx, T_i) + RU(Rx, T_i))$. If the resulting value drops below *minutil* during the mining process, the *PUL* construction for *Rxy* is terminated (as per LA-Prune property, the itemset $Rxy \notin HUI$).

The foregoing description presents the keys steps of the algorithm including the pruning strategies employed for limiting the *HUI* search space. We conduct several experiments next to demonstrate the utility of HUP-Miner.

6. Experimental results

We implemented the proposed algorithm by extending the open-source java library (Fournier-Viger, Gomariz, Soltani, Lam, & Gueniche, 2014). All our experiments were conducted on a Intel Core i5–2500 machine, 3.3 GHz CPU with 4 GB of memory, and running on a Windows OS. In order to ensure robustness of the results, we ran all our experiments three times and report the average results. The results were also assessed for its statistical validity at 5% significance level.

6.1. Experimental design

We evaluated the performance of HUP-Miner on five real-life and three synthetic datasets. The real-life dataset *Chain* was downloaded from NU-MineBench 2.0 (Pisharath et al., 2005) and other four real-life datasets were downloaded from Fournier-Viger et al. (2014). The three synthetic datasets were prepared using the benchmark IBM data generator (Almaden, 1994). The detailed characteristics of the actual datasets used in our experiments are provided in Table 5. In line with the past research studies (Ahmed et al., 2009; Liu et al., 2005; Liu & Qu, 2012), the external utilities for items are generated between 0.01 and 10 using a log-normal distribution. The internal utilities for items are randomly generated in the range of 1 to 10.

Table 5
Dataset characteristics

Dataset	#Trans	#Items (I)	Avg Len (A)	Density (A/I)%
Chain	1112949	46086	7.3	0.0158
Kosarak	990002	41270	8.1	0.0196
Retail	88162	16470	10.3	0.0625
T10I4D100K	100000	870	10.1	1.1609
T20I6D100K	99945	980	19.89	2.0296
T40I10D100K	100000	942	39.6	4.2038
Mushroom	8124	119	23	19.327
Chess	3196	75	37	49.333

6.2. Runtime performance of HUP-Miner

We compare the effectiveness of the proposed method against a state-of-the-art HUI-Miner (Liu & Qu, 2012) algorithm. Fig. 3 gives the results of our experiments on six sparse datasets. The total number of partitions (*K*) is set as 512 for the initial set of experiments. The execution time performance results are shown at varying δ levels. It is clearly evident from the charts that the proposed method performs significantly better than the state-of-the-art method.

We further studied the underlying reason for performance improvement by assessing the total number of candidates generated by both the methods. For this experiment, we took the total number of candidates generated by HUI-Miner as the baseline (100%). The ratio of candidates generated by HUP-Miner divided by HUI-Miner is then plotted in Fig. 4. It can be observed that there is a significant drop in % of candidates generated by HUP-Miner at lower δ levels.

We also assessed the impact of memory consumption of both the algorithms. The results of our experiments shown in Fig. 5 indicate that HUP-Miner actually consumes less memory in many of the cases studied. The lower memory consumption of HUP-Miner can be attributed to the early pruning of unpromising candidates and the reduction in total number of candidates evaluated (Fig. 4), even though it maintains additional partition related information for mining. However, the differences in memory consumption is found to be statistically insignificant.

The run time, memory consumption performance and candidate size reduction charts for dense datasets are shown in Figs. 6 and 7. The performance of HUP-Miner was found to be marginally better compared to HUI-Miner. This is quite expected, as in the case of dense datasets, the partition list almost always contain a non-zero value. Therefore, the chances of significant candidate size reductions through pruning is quite limited as evident from Fig. 7. For example, in the case of benchmark mushroom dataset, the maximum observed candidate size reductions were less than 8%.

The above analysis clearly reveals the usefulness of HUP-Miner and its pruning strategies for sparse datasets.

6.3. Effect of changes in total number of partitions

In the initial set of experiments, we used the *K* value as 512. Now, we vary the partition size (*K*) and study its impact on run time and memory consumption. Figs. 8 and 9 show the results of our experiments at three different *K* levels (512, 1024, 4096). As

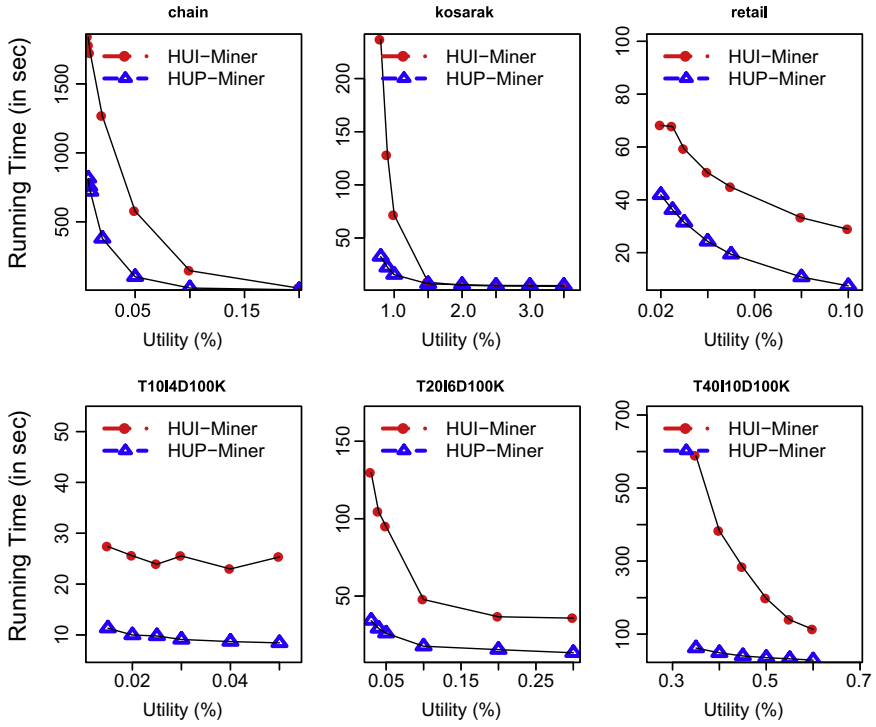


Fig. 3. Run time performance for sparse datasets.

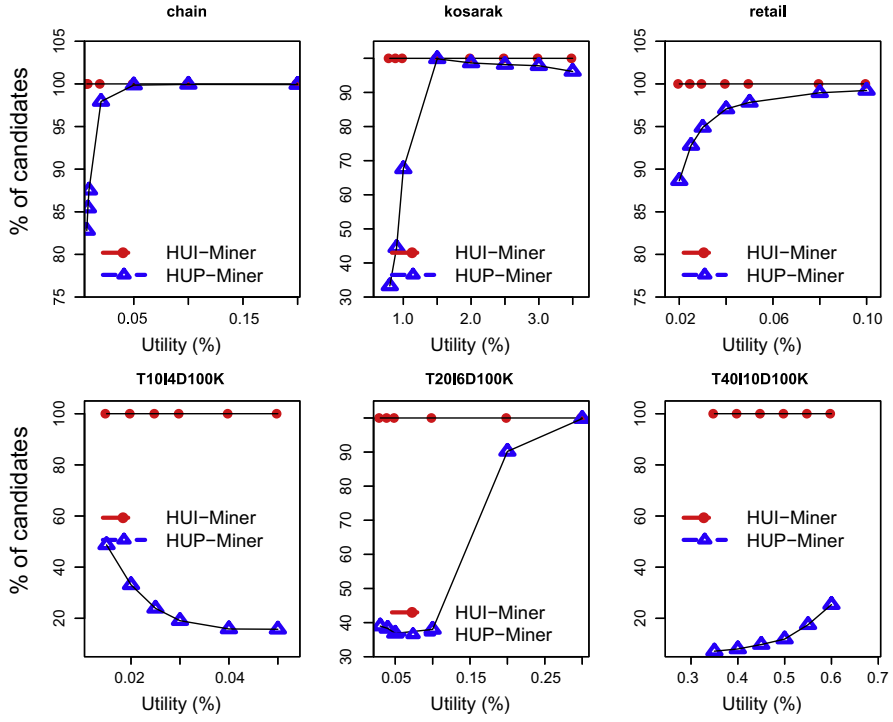


Fig. 4. Percentage reduction in candidates for sparse datasets.

per the results, too high K value leads to increase in run time and/or memory consumption.

It is intuitive that increasing the K value leads to reduction in the total number of transactions within a partition. This helps improve the pruning effectiveness (PU-Prune). However, increasing K to very large value leads to increase in memory requirements for storing the partition information. Furthermore, increase in K incurs additional computational overhead in intersecting large partition lists (PU-Prune). For example, in the case of *retail* dataset, increasing K from 512 to 1024 did not significantly degrade the execution performance. However, further increase of K value to 4096 leads to drop in overall performance. We observed a similar trend for all of the datasets studied.

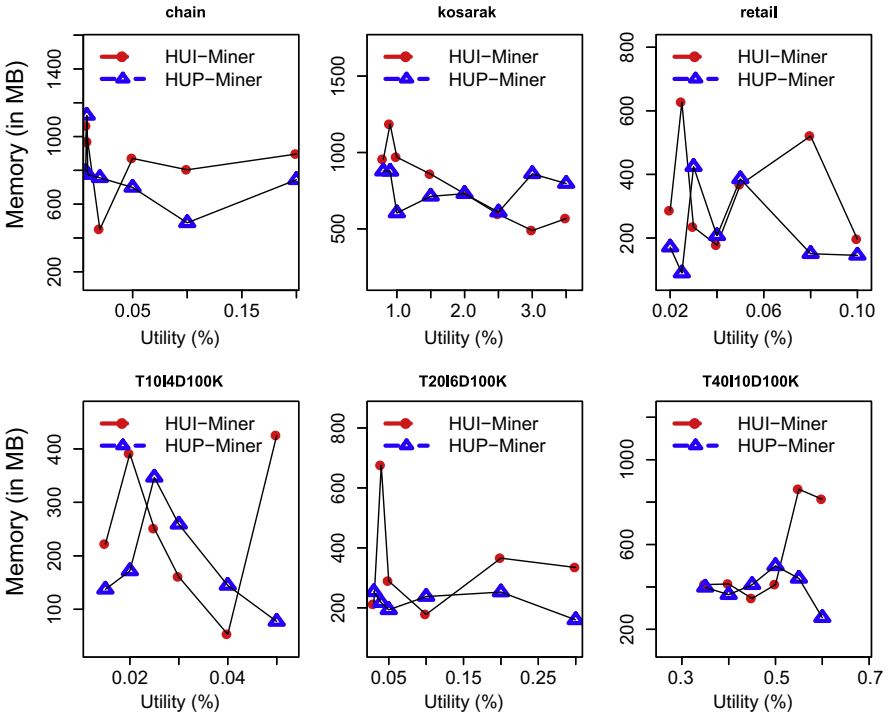


Fig. 5. Memory consumption for sparse datasets.

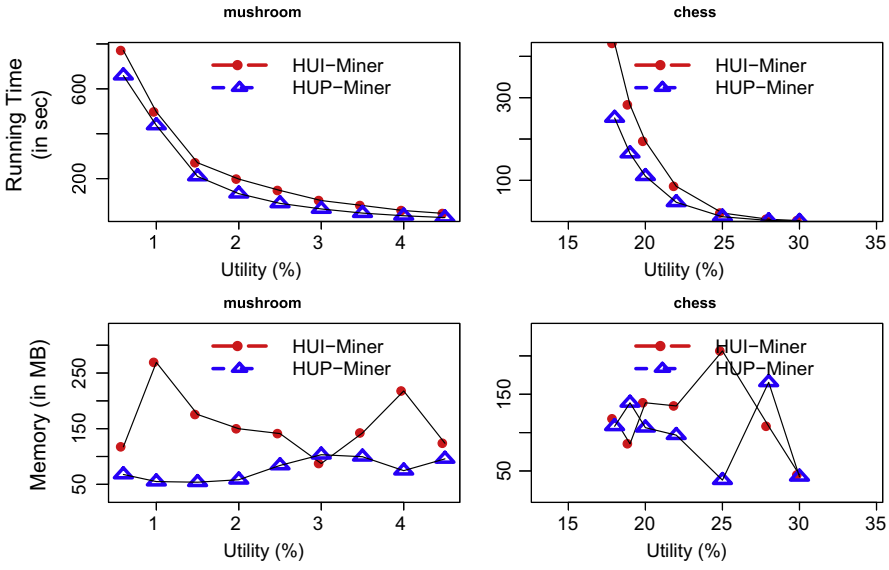


Fig. 6. Run time and memory consumption performance for dense datasets.

6.4. Pruning effectiveness analysis

Fig. 4 gives the candidate size reductions achieved by HUP-Miner over HUI-Miner across multiple δ levels. We further analyze the candidate size reductions at very low δ level for all of the datasets studied. The results of our experiments are presented in Table 6. The columns 3 and 4 give the total number of candidates evaluated by HUP-Miner and % reduction in candidate evaluations over HUI-Miner. The maximum drop in candidate evaluations was observed for the *T40I10D100K* dataset (close to 93%). Similarly, the least drop in candidate evaluations were observed for the *chess* dataset (less than 1%). This is in line with our expectation as the density of the *chess* dataset is very high (close to 50%). The drop in total number of candidate evaluations directly reflect in the run-time performance of HUP-Miner.

Column 5 in Table 6 gives the total number of tidlist generated by HUP-Miner. Recall that *PUL* maintains information about tidlist and partition list that are generated during the *PUL* construction process. In the proposed LA-Prune pruning strategy, the expensive tidlists generation process is curtailed by performing a lookahead pruning. Column 6 presents the results of the effectiveness of LA-Prune in terms of the % reductions in tidlist over the HUI-Miner algorithm. The % reductions are quite high for the sparse datasets but not quite significant for more dense datasets (like *chess*).

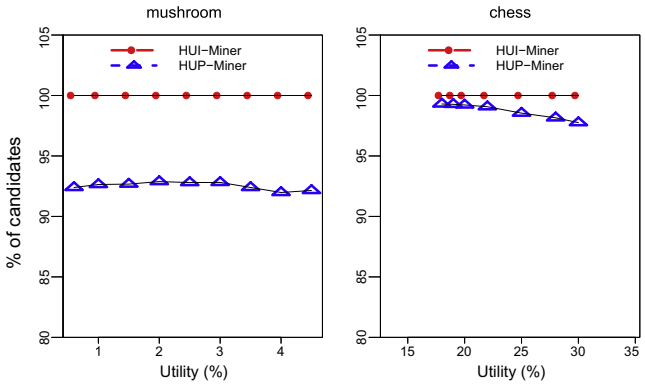


Fig. 7. Percentage reduction in candidates for dense datasets.

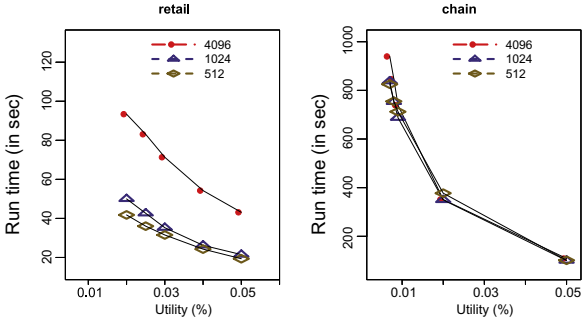


Fig. 8. Effect of changes in K on run time performance.

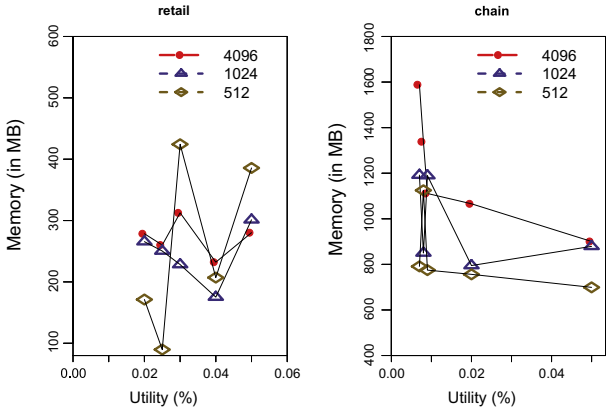


Fig. 9. Effect of changes in K on memory consumption.

Table 6
Performance analysis of candidate sizes.

Dataset	δ	#cand (million)	% Drop (in #cand)	#tidlist (million)	% Drop (in #tidlist)
chain	0.007	192.41	17.20	57.71	30.02
kosarak	0.8	1.14	66.75	112.34	77.07
retail	0.02	32.74	11.35	5.90	30.01
T10I4D100K	0.015	3.80	51.28	12.31	48.24
T20I6D100K	0.03	14.34	61.03	72.39	63.68
T40I10D100K	0.35	1.34	92.85	500.21	73.26
mushroom	0.6	77.74	7.61	12835.75	0.59
chess	18	2.73	0.70	3891.57	1.53

Fig. 10 presents the effect of individual pruning strategies on overall execution performance of HUP-Miner. The relative performance of LA-Prune and PU-Prune was found to vary from one dataset to another. The overall best performance, however, was observed for the combined pruning strategy for all of the datasets studied. The above analysis further corroborates the utility of the two new pruning strategies proposed in this paper.

6.5. Scalability experiments

In the next set of experiments, we conducted scalability tests to assess the utility of HUP-Miner. Fig. 11 provides the results of our experiment on two synthetic benchmark datasets generated using IBM data generator (Almaden, 1994). We configured the data generator to set the total number of items to 100 K and the transaction size to 300 K and 500 K. The results clearly reveals the significant scale up in performance that HUP-Miner achieves over HUI-Miner, especially at lower δ levels. It is evident that the margin of performance difference is likely to increase further as we scale the number of items and the transaction size.

6.6. Discussion

HUP-Miner utilizes four pruning strategies, viz., TWU Prune, U-Prune, PU-Prune and LA-Prune, to improve the efficiency of high utility mining. The PU-Prune and LA-Prune strategies introduced in this paper was found to be effective in limiting the search space of utility mining. Our method was compared against the state-of-the-art HUI-Miner (Liu & Qu, 2012) and the latter method was proven to be superior compared to Two-Phase (Liu et al., 2005), IHUP (Ahmed et al., 2009), UP-Growth (Tseng et al., 2010) and

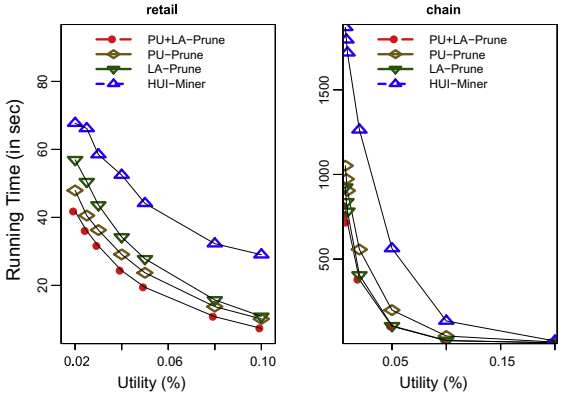


Fig. 10. Effect of pruning strategies.

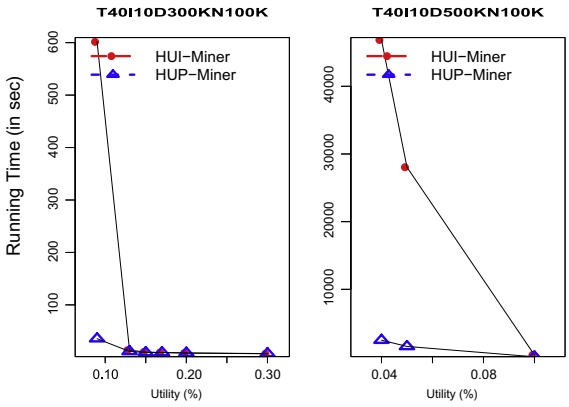


Fig. 11. Scalability experiments.

UP-Growth+ (Tseng et al., 2012). The authors (Liu & Qu, 2012) demonstrate that HUI-Miner is almost up to two orders of magnitude faster than UP-Growth+. Another more recent algorithm (d^2HUP) by Liu et al. (2012) achieves up to an order of magnitude improvement over UP-Growth.

Our method utilizes a new data structure called partitioned utility list (*PUL*) that maintains utility information at the partition level. The PU-Prune strategy that leverages partitioned utility information was found to work quite well, especially for sparse datasets. It was also observed that the additional partitioning information maintained by HUP-Miner do not significantly increase its memory requirements. This is primarily due to the increased pruning efficiency of HUP-Miner.

Another key idea introduced in this paper is the LA-Prune strategy. This pruning strategy primarily aims at limiting the total number of transaction identifier lists (tidlists) generated during the *PUL* construction process. The LA-Prune strategy was found to be effective as evident from the results in Table 6 and the overall performance of HUP-Miner. The relative performance of individual pruning strategy (Fig. 10) also shows promising results.

The choice of the size of the partition (K) can influence the HUP-Miner results. A sensitivity analysis was conducted to assess the impact of changes in K. It was observed that a K value of 512 works best for most of the datasets studied. In future, we intend to determine an optimal value of K that maximizes the performance of individual datasets. In this research work, our primary focus was on designing a new data structure (*PUL*) and devising pruning strategies for efficiently mining *HUIs*. Future research work can consider storing the partition list's identifier information in a compressed bit vector form. This would enable fast intersection of partition lists while performing PU-Prune.

7. Conclusion and future research directions

This paper introduced a new utility mining method for efficiently mining high utility itemsets. The method utilized a partitioned utility list data structure that maintains utility information at a granular partition level. The two new pruning strategies introduced in this paper (PU-Prune and LA-Prune) was found to be effective in limiting the search space of mining. HUP-Miner was observed to be superior compared to the state-of-the-art method on several real and synthetic benchmark datasets. Overall, this paper makes an useful contribution to the literature on mining high utility itemsets.

As a part of future work, we intend to consider the following interesting extensions. First, the lookahead pruning (LA-Prune) can be further optimized to perform batch lookups instead of transaction level lookups. Second, the proposed method can be integrated with other complementary pruning strategies like lookahead closure (Liu et al., 2012) to achieve performance improvements even for dense datasets. Third, the HUP-Miner can be extended to support sequential utility patterns as well as on-shelf utility patterns (Lan et al., 2011).

References

Agrawal, R., Srikant, R. et al. (1994). Fast algorithms for mining association rules. In *Proc. 20th int. conf. very large data bases volume 1215* (pp. 487–499). VLDB.

Ahmed, C. F., Tanbeer, S. K., Jeong, B.-S., & Lee, Y.-K. (2009). Efficient tree structures for high utility pattern mining in incremental databases. *IEEE Transactions on Knowledge and Data Engineering, 21*, 1708–1721.

Ahmed, C. F., Tanbeer, S. K., Jeong, B.-S., & Lee, Y.-K. (2011). Huc-prune: An efficient candidate pruning technique to mine high utility patterns. *Applied Intelligence, 34*, 181–198.

Almaden, I. (1994). Quest synthetic data generation code, http://www.cs.loyola.edu/cgiannel/assoc_gen.html.

Erwin, A., Gopalan, R. P., & Achuthan, N. (2007). A bottom-up projection based algorithm for mining high utility itemsets. *Proceedings of the 2nd international workshop on Integrating artificial intelligence and data mining* (vol.84, pp. 3–11). Australian Computer Society, Inc..

Fournier-Viger, P., Gomariz, A., Soltani, A., Lam, H., & Gueniche, T. (2014). Spmf: Open-source data mining platform, http://www.philippe-fournier-viger.com/spmf.

Lan, G.-C., Hong, T.-P., & Tseng, V. S. (2011). Discovery of high utility itemsets from on-shelf time periods of products. *Expert Systems with Applications, 38*, 5851–5857.

Lan, G.-C., Hong, T.-P., & Tseng, V. S. (2014). An efficient projection-based indexing approach for mining high utility itemsets. *Knowledge and Information Systems, 38*, 85–107.

Lan, G.-C., Hong, T.-P., Tseng, V. S., et al. (2012). An efficient gradual pruning technique for utility mining. *International Journal of Innovative Computing Information and Control, 8*, 5165–5178.

Li, Y.-C., Yeh, J.-S., & Chang, C.-C. (2008). Isolated items discarding strategy for discovering high utility itemsets. *Data & Knowledge Engineering, 64*, 198–217.

Liu, J., Wang, K., & Fung, B. (2012). Direct discovery of high utility itemsets without candidate generation. In *Proceedings of the 2012 IEEE 12th international conference on data mining* (pp. 984–989). IEEE Computer Society.

Liu, M., & Qu, J. (2012). Mining high utility itemsets without candidate generation. In *Proceedings of the 21st ACM international conference on information and knowledge management CIKM '12* (pp. 55–64). New York, NY, USA: ACM.

Liu, Y., Liao, W.-k., & Choudhary, A. (2005). A two-phase algorithm for fast discovery of high utility itemsets. In T. Ho, D. Cheung, & H. Liu (Eds.), *Advances in Knowledge Discovery and Data Mining*(pp. 689–695). volume 3518 of Lecture Notes in Computer Science.

Pisharath, J., Liu, Y., Liao, W.-k., Choudhary, A., Memik, G., & Parhi, J. (2005). Nu-minebench 2.0. *Department of Electrical and Computer Engineering*, Northwestern University, Tech. Rep,.

Tseng, V. S., Shie, B.-E., Wu, C.-W., & Yu, P. S. (2012). Efficient algorithms for mining high utility itemsets from transactional databases. *IEEE Transactions on Knowledge and Data Engineering, 25*, 1772–1786. http://dx.doi.org/10.1109/TKDE.2012.59.

Tseng, V. S., Wu, C.-W., Shie, B.-E., & Yu, P. S. (2010). Up-growth: an efficient algorithm for high utility itemset mining. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 253–262). ACM.

Yao, H., & Hamilton, H. J. (2006). Mining itemset utilities from transaction databases. *Data & Knowledge Engineering, 59*, 603–626.

Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering, 12*, 372–390.